



OTTO VON GUERICKE
UNIVERSITÄT
MAGDEBURG



FAKULTÄT FÜR
INFORMATIK

Master Thesis

Pre-trained Deep Learning Networks for Hyperspectral Image Data

Author: Gowtham Premkumar

Magdeburg, March 27, 2026

Supervisor: Prof. Dr. rer. nat. Frank Ortmeier

Adviser 1: Dr.-Ing. Andreas Herzog

Adviser 2: M.Sc. Konstantin Kirchheim

Premkumar, Gowtham:

Pre-trained Deep Learning Networks for Hyperspectral Image Data

Master Thesis, Otto von Guericke University

Magdeburg, 2025

Abstract

Hyperspectral imaging has emerged as a powerful tool for agricultural applications, particularly in crop quality assessment and variety classification. However, the deployment of deep learning models for hyperspectral data analysis faces significant challenges due to limited labeled datasets and the high dimensionality of spectral information. Self-supervised pre-training methods have proven effective in learning robust feature representations from unlabeled data, yet their application to hyperspectral imaging remains underexplored. This thesis introduces a novel Masked Spatial-Spectral Transformer (MaskedSST) architecture that integrates blockwise 3D patch embedding, factorized attention mechanisms, and self-supervised pre-training strategies to address these challenges. The model employs two distinct pre-training objectives: Masked Autoencoding (MAE) with 85% random token masking and Contrastive Learning (SimCLR) using data augmentation. The architecture is built on a factorized transformer encoder with 4 layers, 8 attention heads, and 128-dimensional embeddings, comprising 1,652,609 trainable parameters. Pre-training was conducted on a diverse multi-domain dataset of 4,512 hyperspectral patches spanning six application domains (coffee, apple leaves, paper, sugar compounds, pottery, and grapes) to learn domain-invariant representations. The model was implemented on a high-performance computing cluster using PyTorch with NVIDIA V100 GPUs. Evaluation on downstream coffee variety classification demonstrated that MAE pre-training achieved X% accuracy with linear probing, outperforming training from scratch by Y%. The blockwise spectral embedding strategy reduced computational complexity by $256\times$ compared to standard self-attention while preserving spectral fidelity. The factorized attention mechanism enabled efficient processing of 4,096 tokens per sample without requiring aggressive downsampling. Limitations include the requirement for substantial pre-training computational resources (200 epochs on multi-GPU infrastructure) and sensitivity to spectral calibration quality across heterogeneous data sources. The model demonstrates that self-supervised pre-training on diverse hyperspectral data yields transferable representations. Future enhancements include exploring cross-domain adaptation techniques, investigating optimal masking strategies for spectral continuity, and extending evaluation to additional crops and imaging modalities for broader applicability in precision agriculture.

Acknowledgments

Contents

List of Figures	ix
Listings	xi
List of Tables	xiii
List of Abbreviations	xv
1 Introduction	1
1.1 Motivation	1
1.2 Main Contributions	2
1.3 Research Questions	2
1.4 Thesis Structure	3
2 Background	5
2.1 Fundamentals of Hyperspectral Imaging	5
2.2 Deep Learning for Hyperspectral Image	7
2.3 Vision Transformer Architecture	8
2.3.1 Patches Formation	8
2.3.2 Patch Flattening and Linear Projection.	9
2.3.3 Positional Encoding	9
2.3.4 Transformer Encoder	10
2.4 Self-Supervised Learning for Vision	12
2.4.1 Masked Autoencoding	12
2.4.2 SimCLR	14
2.5 Evaluation Protocols for Pre-trained Models	14
2.5.1 Linear Probing:	14
2.5.2 Fine-tuning:	15
2.5.3 Convergence Speed Analysis:	16
2.5.4 Feature Visualization:	16
2.5.5 Evaluation Metrics:	16
3 Related Work	17
3.1 Deep Learning for Hyperspectral Image Classification	17
3.2 Transformer Architectures for Hyperspectral Data	18
3.3 Self-Supervised Learning in Computer Vision	18
3.4 Research Gap	19

4	Design and Development	21
4.1	Methodology	21
4.2	Dataset and Data Preprocessing	22
4.2.1	Multi-Domain Pre-training Dataset	22
4.2.2	Data Calibration and Standardization	23
4.2.2.1	Calibration Pipeline	23
4.2.3	Patch Extraction and Size Selection	23
4.2.4	Data Format and Preprocessing	24
4.3	Model Architecture	24
4.3.1	Blockwise 3D Patch Embedding	25
4.3.2	Learnable 3D Positional Encoding	26
4.3.3	Random Token Masking	27
4.3.4	Factorized Spatial-Spectral Transformer Encoder	28
4.3.4.1	Spatial Attention	28
4.3.4.2	Spectral Attention	28
4.3.5	Task-Specific Output Heads	29
4.3.5.1	Reconstruction Head (MAE Pre-training)	29
4.3.5.2	Classification Head (Fine-tuning)	30
4.3.5.3	Contrastive Projection Head (SimCLR Pre-training)	30
4.3.6	Model Summary and Hyperparameters	30
5	Implementation	33
5.1	Hardware Environment	33
5.2	Distributed Training Setup	33
5.3	Optimization and Learning Rate Schedule	34
5.4	Training Configuration	34
5.5	Reproducibility	34
6	Evaluation and Discussion	35
6.1	Experimental Setup	35
6.1.1	Downstream Dataset	35
6.1.2	Pre-training Dataset	35
6.1.3	Experimental Conditions	35
6.2	Pre-training Convergence	36
6.2.1	MAE Pre-training Loss	36
6.2.2	SimCLR Pre-training Loss	37
6.3	Patch Size - 2×2 vs 4×4	39
6.4	Comparison with Existing Baselines	39
7	Conclusion and Future Work	41
	Bibliography	43

List of Figures

4.1	Blockwise 3D Patch Embedding Strategy. Input spectral cube is divided into spatial-spectral patches, and each spectral block is projected by a dedicated learnable weight matrix W_c	25
6.1	mae 2*2	36
6.2	mae 4*4	37
6.3	con 2*2	38
6.4	con 4*4	38

Listings

List of Tables

4.1	MaskedSST Hyperparameters	31
5.1	Optimization Hyperparameters	34
6.1	Distribution of Disease Classes in Training and Testing Sets	35
6.2	Experimental Configuration for Pre-training and Fine-tuning	36
6.3	Summary of Training and Validation Loss Metrics	36
6.4	Summary of Training and Validation Loss Metrics	37
6.5	Accuracy Comparison by Pre-training Method and Patch Size	39
6.6	Accuracy Comparison and Improvement Over Random Baseline	39
6.7	Computational Complexity Comparison by Patch Size	39

List of Abbreviations

1

Introduction

This chapter provides an overview of the motivation behind our work and an initial discussion of the concept of hybrid caching for Large Language Models (LLMs) and its impact on improving the performance of real-time Natural Language Processing (NLP) applications. Moreover, we explore current developments in caching strategies for LLMs and define the goals of this thesis by defining our research questions. We conclude this chapter by providing a synopsis of the thesis structure. The chapter follows the structure mentioned below:

- In Section 1.1, we provide the motivation for the work done in this thesis and concisely discuss hybrid caching while exploring current advancements in the field.
- In Section 1.2, we present our major contributions to research.
- Section 1.3 outlines the research questions that we seek to address in our work.
- In the last section, we describe the structure of the thesis and provide a concise overview of each subsequent section.

1.1 Motivation

Hyperspectral images have recently gained popularity due to its usage in precision agriculture [1], enabling detailed material characterization through the acquisition of spectral information across hundreds of wavelengths bands. Machine learning methods like deep neural networks have become the standard approach for any task using hyperspectral images [10]. However, current practice suffers from a critical limitation, for each new classification task, training must start from scratch. Every new application—whether classifying a different crop type, identifying new material types, or analyzing data from a different sensor—requires collecting a completely new dataset and training an entirely new model from random weight initialization. This approach demands substantial labeled training data and computational resources for every task.

Training deep learning models from scratch for each task requires substantial computational and data. Pretraining a vision transformer [4] for example for 100-200 epochs takes 6 - 10 hr on multi GPU cluster. Repeating this process for every new task costs linearly. The core concept: Instead of training separate models from scratch we can train a general purpose hyperspectral feature extractor once on a large collection of unlabelled hyperspectral images. This Pre trained network learns fundamental spatial- spectral space patterns

of neighbouring pixel relations, spectral band correlation, texture and structure that are common across different hyperspectral tasks. We can use this pre-trained feature extractor as a base for any type of machine learning task and then fine tune it for any downstream tasks [9].

This Thesis investigates the development of Pre-trained deep learning networks for analysing hyperspectral image data. We employ a state-of-the-art Vision Transformer architecture with factorized spatial-spectral attention mechanisms, designed specifically to process high-dimensional hyperspectral data and also compare two prominent self-supervised learning methods are compared: Masked Autoencoding (MAE) [5], which learns representations by reconstructing randomly masked image patches, and Contrastive Learning (SimCLR) [2], which learns discriminative features by distinguishing augmented views of the same image from different samples. Our evaluation on a classification shows which approach produces features that transfer most effectively to downstream classification task. An overview of the challenges can be found in Figure 2.1. Among these challenges, this thesis focuses on inference latency and resource utilization.

1.2 Main Contributions

The research led to the following contributions made during this thesis work, which are listed below:

1. We implemented a Vision Transformer architecture with spatial and spectral attention mechanism specifically tailored for processing high-dimensional hyperspectral data with 256 spectral bands.
2. We compare two state of the art self - supervised pre-training methods - Masked Autoencoding (MAE) and Contrastive Learning (SimCLR).
3. We demonstrate that using pre-trained model have significant advantage.

1.3 Research Questions

The research questions defined here guide the investigation of self-supervised pre-training strategies applied to hyperspectral image tasks with limited labelled data. [11] Introduced Masked Vision Transformer architecture specifically designed for hyperspectral image, which showed effectiveness of Masked Autoencoding for learning spatial - spectral features. However this work exclusively focused on reconstruction-based learning through MAE, leaving the alternative approaches unexplored with the same architecture. We investigate whether Contrastive learning as an alternative strategy. We investigate whether this learning objective which optimises by discrimination rather than reconstruction better aligns with our downstream classification tasks

Thus, the following research questions are identified to guide this study :

1. Can self-supervised pre-training improve hyperspectral image classification performance in small-data regimes compared to training from random initialization?
2. Which self-supervised pre-training approach is more effective for hyperspectral classification: reconstruction-based learning (Masked Autoencoding) or discriminative learning (Contrastive Learning)?
3. Do the learned representations from self-supervised pre-training capture meaningful spatial-spectral patterns that transfer effectively to downstream classification tasks?

1.4 Thesis Structure

The thesis has the following structure:

- Chapter 2 provides an overview of the fundamental concepts of different topics and the background information for methods used in this work.
- In Chapter 3, we provide information about our contributions.
- Chapter 4 provides a detailed overview of the conducted experiments and the hardware and software technologies used to produce the results.
- Chapter 5 discusses the results of our evaluation.
- Chapter 6 provides the conclusion of the thesis and a discussion of possible future work.
- The last section contains the bibliography and appendices used throughout our thesis work.

2

Background

In this chapter, we discuss the essential concepts that are relevant to our work. We provide an overview of hyperspectral image classification, self-supervised learning methods, and Vision Transformer architectures. Then there is also a discussion on specific self-supervised strategies—Masked Autoencoding and Contrastive Learning—and their application to hyperspectral data. We then discuss the role of pre-training in addressing limited labeled data challenges and review evaluation protocols for measuring feature transferability. The chapter has the following structure:

- In Section 2.1, we start with a discussion on hyperspectral image classification by providing a detailed overview of traditional machine learning and deep learning approaches. We discuss the key challenges of high dimensionality and limited labeled data while reviewing relevant studies related to agricultural applications and coffee classification.
- In Section 2.2, we discuss the fundamentals of self-supervised learning, contrasting reconstruction-based and contrastive learning paradigms. We review successful applications in computer vision and identify the research gap in hyperspectral imaging.
- In Section 2.3, we introduce Vision Transformers and their adaptation to hyperspectral data, focusing on the Masked Vision Transformer architecture by Scheibenreif et al. [cite] that forms the basis of our work. We discuss spatial-spectral attention mechanisms and architectural design choices.
- In Section 2.4, we review Masked Autoencoding (MAE) and Contrastive Learning (SimCLR) in detail, explaining their training objectives, advantages, and challenges for hyperspectral applications.
- In Section 2.5, we discuss evaluation protocols including linear probing, fine-tuning strategies, and metrics for assessing feature quality and transfer learning effectiveness.
- In Section 2.6, we provide a summary of the chapter and identify the specific gap our thesis addresses.

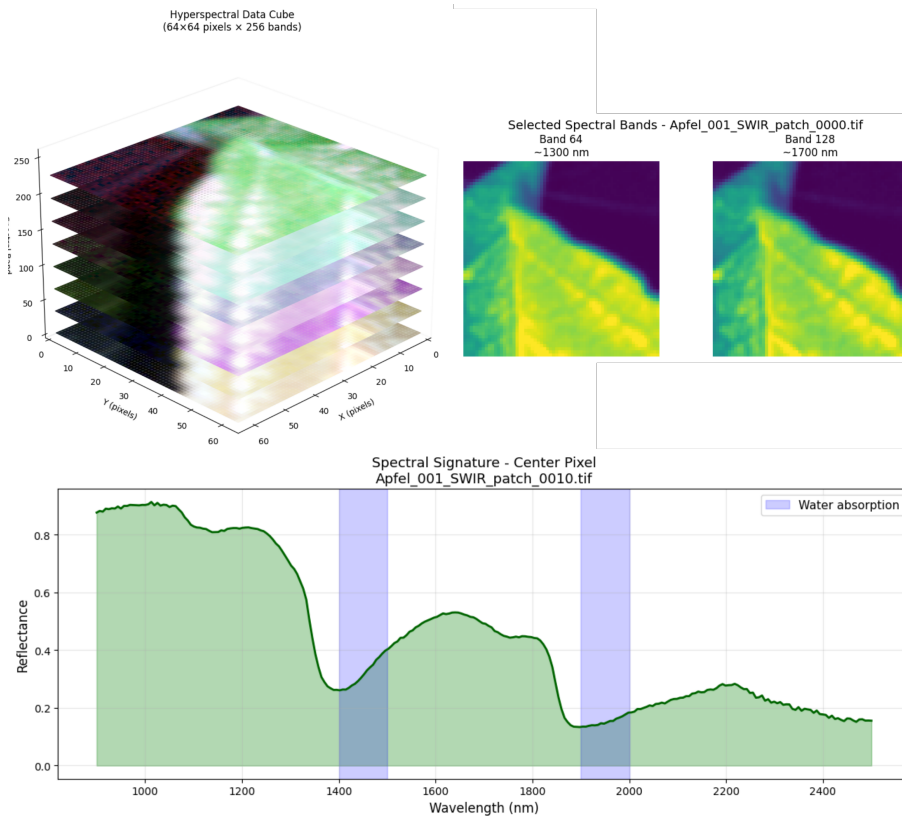
2.1 Fundamentals of Hyperspectral Imaging

Hyperspectral Imagery is an advanced sensing technology that stores detailed information across hundreds of contiguous wavelengths and is one of the most significant discoveries

in remote sensing imaging technology. Unlike conventional RGB sensors which capture only 3 broad color channels (red, green, blue) which lie in visible spectrum (400-700 nm), hyperspectral sensors acquire spectral measurements across 100 to 300 + color channels with narrow bandwidths (typically 5-10 nm), spanning wavelengths from the visible (VIS) through near-infrared (NIR, 700-1000 nm) to short-wave infrared (SWIR, 1000-2500 nm) regions of the electromagnetic spectrum.

Hyperspectral Data Structure - Hyperspectral images create a rich 3 dimensional data structure called a hyperspectral cube, which can be represented in mathematical form as $I \in \mathbb{R}^{H \times W \times C}$, where H and W denote the spatial height and width in pixels, and C represents the number of spectral bands. Each pixel at a spatial location (x,y) contains a spectral signature $s(x,y)=[r_1,r_2,...,r_C]$ where r_i represents the reflectance or radiance value at wavelength λ_i .

This spectral signature is a unique electromagnetic "fingerprint" that characterizes the material properties at that spatial location. For example an typical hyperspectral image of 512×512 pixels and 256 spectral bands contains over 67 million individual measurements ($512 \times 512 \times 256 = 67,108,864$ values) which provides detailed information of the composition of the target space.



Spectral Signatures and Material Identification - The fundamental principle of capturing reflectance of light at different wavelengths is that each material has different composition,

chemical bonds and physical structure and pattern which is unique to this substance. Visible and Near-Infrared (VNIR) sensors operate in the wavelength range of 400-1000 nm, covering the visible spectrum (400-700 nm) and extending into the near-infrared region. Short-Wave Infrared (SWIR) sensors operate in the 1000-2500 nm wavelength range, providing complementary information about molecular composition, moisture content, and chemical properties invisible to VNIR sensors. For instance, Different coffee varieties—Arabica and Robusta—differ in their chemical composition (caffeine content, lipid profiles, organic acids), producing subtle but measurable differences in their SWIR spectral signatures that are imperceptible to RGB cameras but detectable with hyperspectral sensors.

Data Acquisition and Challenges - Hyperspectral imaging presents significant challenges. The sensors are expensive, often costing tens to hundreds of thousands of dollars, limiting accessibility. Data collection requires careful calibration, atmospheric correction for outdoor sensing and controlled illumination. Then there is the size of HIS data as they can be several megabytes to gigabytes easily their storage, transmission and processing becomes a major challenge. Most critically for machine learning applications, acquiring labeled training data is expensive and time-consuming, requiring domain expertise to annotate materials, species, or conditions based on spectral signatures rather than visual appearance alone.

2.2 Deep Learning for Hyperspectral Image

Deep learning techniques have revolutionized HSI analysis by learning complex relations between features from raw spectral data.

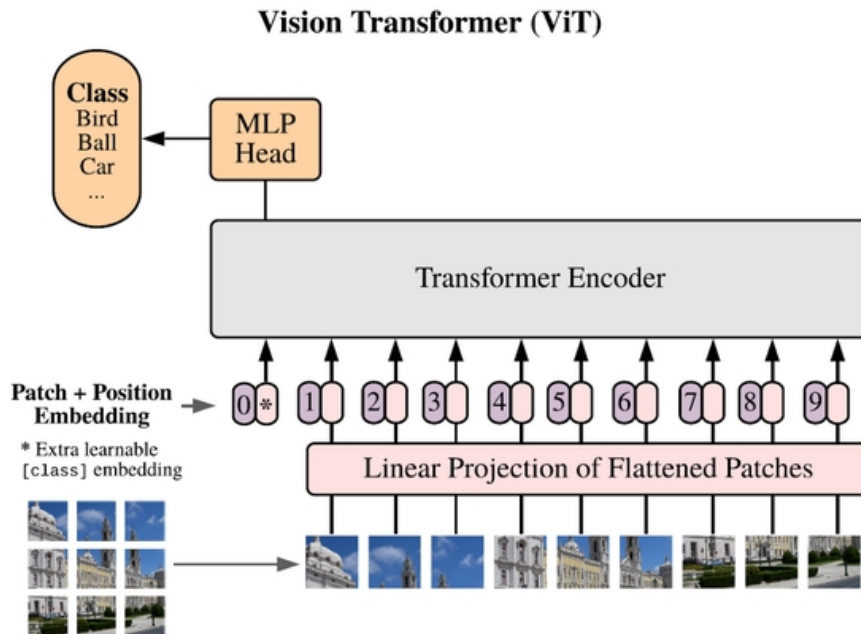
- **Spectral-Only Processing** - [7] introduced 1D CNNs that treat each pixel's spectral signature as a sequence, it applied convolution along the spectral dimension to learn features. While computationally efficient and effectively capturing patterns and spectral correlations, it could not leverage spatial relations like relations between neighboring pixels which often contained valuable information.
- **Spatial-Only Processing** - [3] applied 2D CNNs to spatial neighborhoods extracted from HSI, this method used spectral bands independently or reducing them using PCA before processing. This worked well in preserving spatial structure like edges, texture and local patterns well but as an independent features rather than capturing its correlation and employing reduction steps like pca reduced spectral information which are crucial for material discrimination
- **Combination of Spatial and Spectral Learning** - Recognizing these complementary limitations, [8] proposed 3D CNNs that process both spatial and spectral dimensions using 3D convolutional kernels. A 3D kernel of size $(k \times k \times d)$ captures local

spatial neighborhoods ($k \times k$ pixels) across d consecutive spectral bands, learning filters that respond to specific spatial-spectral patterns. But this convolutions uses a large amount of computational overhead and the memory requirement proportional to number of bands, and training such deep learning models required careful hyperparameter tuning so that the model does not overfit.

- **Transformer Architectures for Hyperspectral Data** - The success of transformer architectures on natural images has motivated its use on hyperspectral analysis [6] introduced SpectralFormer, unlike the CNN's the transformers compute global attention which allows spectral a bands to attends to all other bands and this global modeling captures distant spectral correlations that Convolutions miss. [?].

2.3 Vision Transformer Architecture

The Vision Transformer (ViT) architecture, introduced by Dosovitskiy et al. (2021), this marked the shift in computer vision from other architectures to transformer architecture for image analysis. Unlike the Convolutional Neural Networks (CNNs) that process images through repetitive convolutions, transformers treats the images as patches and apply global attention mechanisms. This allows each image region to directly attend to all other regions regardless of spatial distance, this allows capturing relations and dependencies over long distance.



2.3.1 Patches Formation

The innovation of the vision transformers lies in the simple architecture and usage of the already used transformer model which has proven successful for language modeling. Like

in the transformer model instead of word tokens vision transformer uses image patches, the image of dimensions $H \times W \times C$ (height, width, channels) is treated as N flattened patches.

For example, the first stage converts a 2D image into a sequence of patch embeddings suitable for transformer processing. An input image of size $H \times W \times C$ is divided into non-overlapping patches of fixed size $P \times P$, yielding

$$N = \frac{H \times W}{P^2}$$

patches. To obtain $N = 9$ patches, the image must be partitioned into a 3×3 grid, meaning

$$P = \frac{H}{3}, \quad P = \frac{W}{3}.$$

For instance, an image of size 300×300 with $C = 3$ channels (RGB) divided into 100×100 patches produces $N = 9$ patches, each containing

$$P^2 \times C = 100^2 \times 3 = 30,000$$

values.

2.3.2 Patch Flattening and Linear Projection.

Each $P \times P \times C$ patch is flattened into a single vector of length P^2C . These flattened patch vectors are then linearly projected to a fixed latent dimension D through a learnable projection matrix

$$W_{\text{patch}} \in \mathbb{R}^{(P^2C) \times D}.$$

This projection step plays a few important roles. First, it ensures that every patch—no matter its original size—gets represented using the same dimensionality D . It can also reduce the raw patch dimensionality when $D < P^2C$ (e.g., projecting a 30,000-dimensional raw patch to a 3000-dimensional embedding). Beyond that, the learnable projection matrix helps the model highlight useful or discriminative patterns within each patch. After this transformation, we end up with N patch embeddings, each of size D , forming a sequence matrix

$$X \in \mathbb{R}^{N \times D}.$$

ViT also adds a token called CLS token in the start of the of the patch sequence, it gathers information throughout the self-attention process. After passing through layers of the transformers this CLS token becomes the final image-level representation used for classification

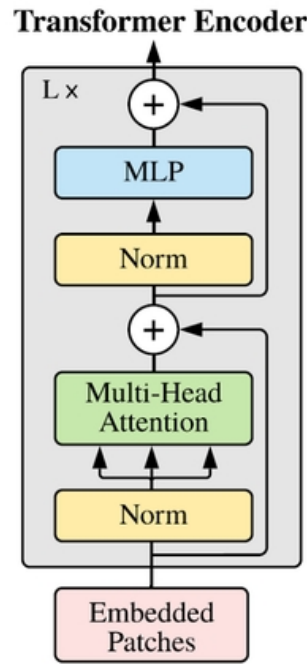
2.3.3 Positional Encoding

ViT includes learnable positional embeddings so that the position of the patch where it comes from is known. These embeddings form a matrix $E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$, with one entry

for every patch and one for the CLS token. These embeddings are added directly to the patch embedding in the input layer and is trained along with the rest of the model. Instead of using a fixed sinusoidal encodings we have learnable encodings so that the transformer can learn the positional patterns, it helps in capturing the 2D layout of the image patches

2.3.4 Transformer Encoder

The Transformer encoder consists of many identical layers stacked one after another. Each block contains two main components: a Multi-Head Self-Attention (MSA) module and a Feed-Forward Network (FFN). Both these submodules are connected with residual connections and followed by layer normalization ensuring stable optimization and effective gradient flow.



Multi-Head Self-Attention (MSA) :

Self-attention enables each token to aggregate information from all other tokens in the sequence, with the model learning how strongly each token should influence the others. Given an input sequence

$$Z \in \mathbb{R}^{(N+1) \times D},$$

the model computes queries, keys, and values using learned projection matrices

$$W_Q, W_K, W_V \in \mathbb{R}^{D \times D_{\text{head}}}.$$

These projections are defined as

$$Q = ZW_Q, \quad K = ZW_K, \quad V = ZW_V.$$

The attention mechanism measures the compatibility between each query and all keys using scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{D_{\text{head}}}}\right)V.$$

The scaling factor $\sqrt{D_{\text{head}}}$ prevents very large dot-product values, which would push the softmax into regions with very small gradients. The softmax operation converts raw similarity scores into normalized attention weights that determine how much each value vector contributes to the output.

Multi-Head Attention :

Instead of computing a single attention operation, the transformer uses h parallel attention heads. Each head has its own set of projection matrices, allowing the model to learn different types of relationships simultaneously. For example, one head may focus on spatially adjacent patches, another on semantically similar regions, and others on more abstract or task-specific patterns.

For head i , the computation is

$$\text{head}_i = \text{Attention}(ZW_{Q_i}, ZW_{K_i}, ZW_{V_i}).$$

The outputs of all heads are concatenated and projected back to the model dimension D using a final output matrix:

$$\text{MSA}(Z) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W_O,$$

where

$$W_O \in \mathbb{R}^{(hD_{\text{head}}) \times D}.$$

This projection mixes information across heads and produces the refined token representations that are passed to the subsequent feed-forward network.

Position-wise Feed-Forward Network (FFN)

After self-attention aggregates information across all tokens, each token representation passes independently through a two-layer feed-forward network. This network applies the same learned transformation to every token position, hence "position-wise." The FFN is defined as:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2,$$

where $W_1 \in \mathbb{R}^{D \times D_{\text{FFN}}}$ expands the embedding dimension to D_{FFN} (typically $4D$), and $W_2 \in \mathbb{R}^{D_{\text{FFN}} \times D}$ projects the representation back to the model dimension. This expansion-compression pattern creates what's called a bottleneck architecture. By forcing information through a wider intermediate representation before compressing it back, the network learns rich, non-linear transformations that go beyond what self-attention alone can cap-

ture. The GELU (Gaussian Error Linear Unit) activation function introduces non-linearity between these two layers. Unlike the sharp cutoff of ReLU, GELU provides a smoother activation curve that tends to work better in transformer architectures.

Residual Connections and Layer Normalization

Both the multi-head attention and feed-forward network components are enhanced with two training stabilization techniques: residual connections and layer normalization. The complete encoder block operates as follows:

$$z'_l = \text{MSA}(\text{LayerNorm}(z_l)) + z_l,$$

$$z_{l+1} = \text{FFN}(\text{LayerNorm}(z'_l)) + z'_l.$$

Residual connections (the “ $+ z_l$ ” and “ $+ z'_l$ ” terms) create shortcut paths that allow gradients to flow directly backward through the network during training. Without these shortcuts, very deep networks suffer from vanishing gradients—the learning signal becomes progressively weaker as it propagates backward through many layers. Residual connections mitigate this issue, enabling transformers to scale to 12, 24, or even more layers while maintaining stable and reliable training.

Layer normalization standardizes the activations across the feature dimension before each sub-layer. This normalization reduces the sensitivity to initialization and helps maintain stable activation magnitudes throughout training. Vision Transformers use what’s called a “pre-norm” configuration, where normalization happens before each operation rather than after (the “post-norm” approach used in the original transformer). This pre-norm design has proven more stable when training very deep vision models.

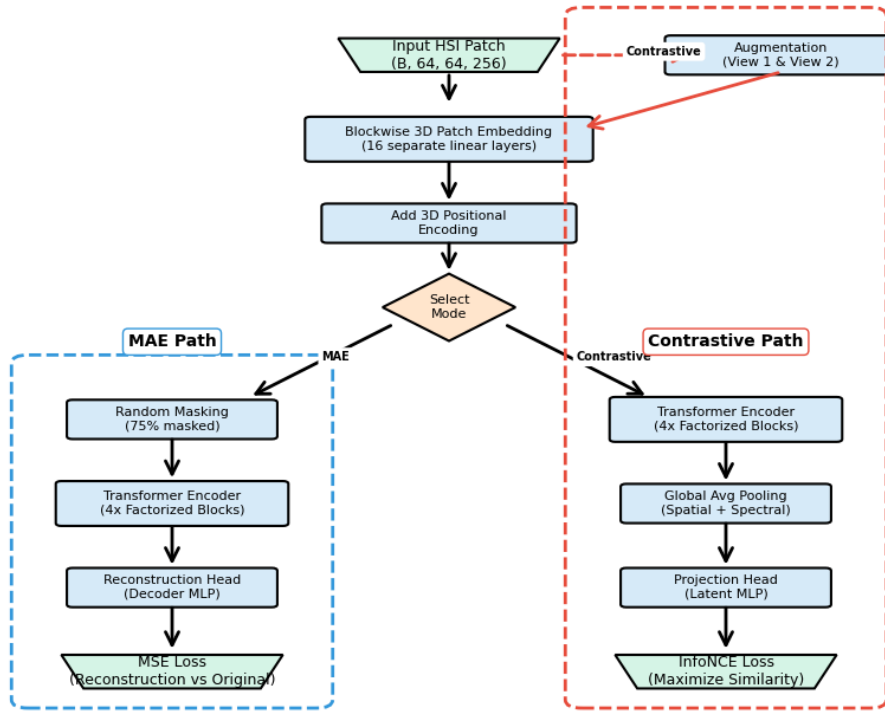
Stacking Encoder Blocks

The encoder consists of L such blocks stacked sequentially, with each block refining the token representations produced by the previous one. Early layers typically capture low-level visual patterns such as edges, textures, and local color variations, while deeper layers learn higher-level semantic structure, including object parts and global contextual relationships. In hyperspectral imagery, this hierarchical progression may extend from basic spectral absorption features to material-specific signatures and, ultimately, semantic object categories.

2.4 Self-Supervised Learning for Vision

2.4.1 Masked Autoencoding

Masked Autoencoding (MAE), introduced by He et al. (2022), learns visual representations by reconstructing randomly masked image regions from the visible context. The method adapts masked language modeling from NLP to vision: if a model can accurately



reconstruct 75–85% of an image from only 15–25% of visible patches, it must capture spatial-spectral patterns and structural relationships.

Architecture. MAE employs an asymmetric encoder-decoder design. The encoder, a Vision Transformer (ViT), processes only the visible patches (15–25% of the image), significantly reducing computational cost because self-attention scales quadratically with sequence length. After encoding, learnable mask tokens are inserted at the masked positions, and a lightweight decoder (1–2 transformer blocks) reconstructs the full image. The reconstruction loss is computed using Mean Squared Error (MSE) between predicted and original masked patches.

Training. During pre-training, each image is randomly masked (with a different mask each iteration). The encoder processes visible patches, and the decoder predicts the masked regions. The loss is computed only on masked patches, forcing the model to learn spatial-spectral correlations rather than trivially copying visible content.

2.4.2 SimCLR

SimCLR (Chen et al. 2020) learns representations by maximizing agreement between augmented views of the same image while minimizing agreement with different images. This discriminative objective aligns naturally with classification tasks: the model learns what makes samples similar or different.

Architecture. SimCLR generates two randomly augmented views of each image, encodes both using a shared Vision Transformer, and applies a projection head (a 2-layer MLP) to map representations into a contrastive embedding space. The NT-Xent (Normalized Temperature-scaled Cross-Entropy) loss pulls positive pairs together while pushing negatives apart:

$$\ell_{i,j} = -\log \left(\frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_k \exp(\text{sim}(z_i, z_k)/\tau)} \right),$$

where $\text{sim}(z_i, z_j)$ denotes cosine similarity and $\tau = 0.5$ is the temperature parameter.

Data Augmentation. Effective contrastive learning requires strong augmentations that preserve semantic content while altering appearance. For hyperspectral data, suitable augmentations include:

- **Spatial:** horizontal/vertical flips, rotations, random crops
- **Spectral:** Gaussian noise, spectral shifts, band dropout, spectral scaling

Training. With a batch size of N , SimCLR creates $2N$ augmented views. Each view has one positive pair (the other augmentation of the same image) and $2N - 2$ negatives (all other images). Larger batch sizes provide more negatives, improving representation quality but requiring substantial GPU memory.

2.5 Evaluation Protocols for Pre-trained Models

Evaluating the quality of self-supervised pre-trained representations requires protocols that measure how effectively learned features transfer to downstream tasks. Unlike supervised learning where validation accuracy directly reflects model quality, self-supervised methods learn without task-specific objectives, necessitating indirect evaluation approaches. This section reviews standard evaluation protocols used to assess pre-trained feature quality.

2.5.1 Linear Probing:

The concept of caching was first introduced by Maurice Wilkes in 1965 [?] in which he called caching "Slave Memory". Caching is the process of temporarily storing frequently used data so that it can be retrieved more quickly when needed without recomputation [?]. In this section, we introduce the key concepts and terminology commonly used in

caching systems.

Linear probing evaluates whether pre-trained features are linearly separable for a target task by freezing the encoder and training only a linear classifier on top. Given a pre-trained encoder $f(\cdot)$ that maps images to D -dimensional representations, linear probing adds a single linear layer $W \in \mathbb{R}^{D \times C}$ that maps representations to C classes:

$$y = \text{softmax}(W^\top f(x) + b).$$

During linear probe evaluation, all encoder parameters remain frozen; only the classification layer (W, b) is trained using labeled data. This protocol tests whether the encoder has already learned discriminative features that separate classes without requiring additional feature learning.

Advantages. Linear probing provides a fair comparison across different pre-training methods by isolating feature quality from optimization effects. It requires minimal labeled data (hundreds rather than thousands of samples) since only a shallow classifier is trained, reducing the risk of overfitting. Training is computationally efficient (minutes to hours rather than days) due to the small number of trainable parameters. High linear probe accuracy indicates that pre-trained features are task-ready and require minimal adaptation.

Interpretation. Strong linear probe performance suggests that the encoder has learned semantically meaningful, discriminative features aligned with the downstream task. Conversely, poor performance indicates that the learned features capture patterns not relevant to the classification objective, implying that substantial fine-tuning may be required for effective adaptation.

2.5.2 Fine-tuning:

Fine-tuning unfreezes the entire pre-trained encoder and updates all parameters on the downstream task with labeled data. Unlike linear probing, fine-tuning allows the encoder to adapt its representations specifically for the target task. Typically, fine-tuning uses smaller learning rates (1-2 orders of magnitude lower than training from scratch) to preserve pre-trained knowledge while making task-specific adjustments.

Comparison with Linear Probing. The performance gap between linear probing and fine-tuning reveals feature adaptability. Small gaps indicate pre-trained features already align well with the task, requiring minimal modification. Large gaps suggest features need substantial adaptation, potentially indicating suboptimal pre-training for the specific downstream application.

2.5.3 Convergence Speed Analysis:

Beyond final accuracy, convergence speed measures how quickly models reach peak performance during training. Pre-trained models with high-quality features converge faster than random initialization, requiring fewer epochs and less labeled data to achieve similar accuracy. Convergence analysis tracks validation accuracy across epochs, identifying:

- Epochs to convergence: Number of training epochs needed to reach 95 % of final accuracy [?].
- Training efficiency: Labeled samples required to achieve target performance [?], [?].
- Gradient dynamics: Stability and magnitude of gradients during early training [?], [?].

2.5.4 Feature Visualization:

Dimensionality reduction techniques visualize learned feature spaces to qualitatively assess cluster structure and class separability. Common approaches include:

- t-SNE (t-Distributed Stochastic Neighbor Embedding): Non-linear projection preserving local neighborhood structure, effective for visualizing cluster formation [?].
- UMAP (Uniform Manifold Approximation and Projection): Preserves both local and global structure, faster than t-SNE for large datasets [?], [?].
- PCA (Principal Component Analysis): Linear projection along maximum variance directions, interpretable but limited for complex distributions [?], [?].

2.5.5 Evaluation Metrics:

3

Related Work

In this chapter we review the existing literature related for the thesis. We organize the discussion into four areas: (1) deep learning methods for hyperspectral image classification, (2) transformer architectures adapted for hyperspectral data, (3) self-supervised pre-training methods in computer vision, and (4) self-supervised learning applied to hyperspectral imaging. We finish by identifying the specific gap this thesis addresses.

3.1 Deep Learning for Hyperspectral Image Classification

Deep learning has become very popular method for hyperspectral image analysis, this by replacing traditional machine learning methods such as Support Vector Machines (SVMs) and Random Forests. Paoletti et al. provide a comprehensive review of deep learning classifiers for hyperspectral imaging, tracing the evolution from simple multi-layer perceptrons to sophisticated convolutional and recurrent architectures. Early deep learning approaches processed hyperspectral data along a single dimension. Hu et al. introduced 1D CNNs that treat each pixel’s spectral signature as a sequential signal, applying convolutions along the spectral dimension to extract features. While his approach is computationally efficient, this approach discards spatial context entirely—neighboring pixels, which often share material properties and exhibit correlated reflectance, are processed independently. This spatial blindness limits classification accuracy in scenarios where spatial texture carries discriminative information.

To include and use the spatial structure, Chen et al. applied 2D CNNs to local spatial neighborhoods extracted around each pixel. By processing spatial patches, these models capture texture patterns, edges, and local geometry. However, spectral dimensionality is typically reduced through PCA or treated as independent channels, discarding spectral correlation that is critical for material discrimination in hyperspectral data.

Li et al. addressed both limitations by proposing 3D CNNs that process spatial and spectral dimensions jointly using volumetric convolutional kernels. A kernel of size $k \times k \times d$ captures local spatial neighborhoods across d consecutive spectral bands, enabling filters to respond to coupled spatial-spectral patterns. Despite this improvement, 3D convolutions introduce significant memory and computational overhead that scales with the number of spectral bands, making them expensive to train on high-dimensional data.

3.2 Transformer Architectures for Hyperspectral Data

The success of transformer architectures in natural language processing motivated their adoption in computer vision. Dosovitskiy et al. introduced the Vision Transformer (ViT), demonstrating that a pure transformer applied to sequences of non-overlapping image patches achieves competitive performance with CNNs on image classification benchmarks. Unlike CNNs, transformers compute global self-attention, enabling each patch to directly attend to all other patches regardless of distance which is particularly valuable for modeling long-range dependencies.

Hong et al. extended this idea to hyperspectral data with SpectralFormer, which applies transformer attention along the spectral dimension to model dependencies between wavelength bands. SpectralFormer demonstrated that global spectral modeling captures distant band correlations that local convolutional operations miss—for example, correlations between visible absorption peaks and SWIR molecular signatures. However, SpectralFormer operates primarily in the spectral dimension and does not fully exploit joint spatial-spectral attention.

Most directly relevant to our work, Scheibenreif et al. proposed a Masked Vision Transformer architecture specifically designed for hyperspectral image classification. Their work introduced factorized spatial-spectral attention and demonstrated the effectiveness of Masked Autoencoding (MAE) for learning spatial-spectral features from hyperspectral data. Their results showed that pre-training with masked reconstruction significantly improves classification accuracy, particularly in low-data regimes. However, this work exclusively investigated the MAE pre-training objective, leaving open the question of whether contrastive learning—an alternative self-supervised strategy—could yield comparable or superior representations. Our thesis directly extends this line of work by implementing and comparing both MAE and SimCLR pre-training strategies on the same architecture and dataset.

3.3 Self-Supervised Learning in Computer Vision

Self-supervised learning has emerged as a powerful method for learning about visual representation of unlabeled data. Liu et al. provide a broad survey of self-supervised methods, distinguishing between generative approaches (which learn by reconstructing inputs) and discriminative approaches (which learn by distinguishing between samples). Both paradigms have proven highly effective at learning transferable representations that reduce the need for labeled data in downstream tasks.

He et al. introduced Masked Autoencoders (MAE), which adapt masked language modeling from NLP to the visual domain. MAE randomly masks a large proportion (75–85 percent) of image patches and trains the model to reconstruct the missing regions from the visible context. The key insight is that a high masking ratio prevents trivial reconstruction by interpolation, forcing the model to learn deep semantic and structural understanding. MAE achieves strong transfer learning performance on image classification,

object detection, and segmentation benchmarks, while being computationally efficient due to processing only the visible (unmasked) tokens in the encoder.

Chen et al. proposed SimCLR, a contrastive learning framework that learns representations by maximizing agreement between differently augmented views of the same image while minimizing agreement with views from different images. SimCLR uses the NT-Xent (Normalized Temperature-scaled Cross-Entropy) loss and relies heavily on strong data augmentation and large batch sizes to provide sufficient negative samples. SimCLR representations transfer effectively to classification tasks because the discriminative objective encourages the model to encode the properties that make samples similar or different—properties that are also relevant for downstream classification.

3.4 Research Gap

Existing work demonstrates the effectiveness of deep learning for hyperspectral classification and the potential of transformer-based architectures for spectral modeling. Self-supervised pre-training has proven powerful for natural image representations, but its systematic application to hyperspectral data remains limited. Scheibenreif et al. represents the closest prior work, demonstrating MAE-based pre-training for hyperspectral transformers, but their study does not investigate contrastive learning as an alternative objective.

This thesis fills this gap by implementing both MAE and SimCLR pre-training strategies within a unified Masked Spatial-Spectral Transformer architecture, enabling a direct empirical comparison of reconstruction-based and discriminative self-supervised objectives for hyperspectral image classification. Furthermore, we pre-train on a diverse multi-domain dataset spanning six application domains to learn domain-invariant representations, which prior work has not explored in this setting.

4

Design and Development

In Chapter 2, the objective was to provide an overview of the important background related to the concepts for our thesis work. This chapter presents the specific information of our work and our approach. The chapter contains the following sections:

- In the next section, we provide the methodology used to address the research questions as stated in Section 1.3.
- Section 4.2 describes the overview of the coffee hyperspectral dataset, including data format, structure, and preprocessing pipeline applied to prepare the data for model training.
- Section 4.3 describes the Vision Transformer architecture with factorized spatial-spectral attention mechanisms, including 3D patch embedding, positional encoding, and transformer encoder blocks.
- Section 4.4 describes the two self-supervised pre-training strategies implemented: Masked Autoencoding (MAE) and Contrastive Learning (SimCLR), including their training objectives, loss functions, and data augmentation techniques for hyperspectral data.
- Section 4.5 describes the evaluation protocol for assessing pre-trained model quality, including linear probe evaluation and the random initialization baseline for performance comparison.
- Section 4.6 outlines the hardware and software specifications, training configurations, and hyperparameters used to implement and evaluate the pre-trained models.
- The last section provides the summary of the discussions and work presented in this chapter.

4.1 Methodology

This section outlines our approach to addressing the research questions posed in Section 1.3. Our methodology follows a systematic workflow: we first aggregate a large-scale, multi-domain hyperspectral dataset to enable robust self-supervised learning, applying necessary calibration and preprocessing to standardize diverse data sources. We then implement a Vision Transformer architecture adapted for hyperspectral data and apply two

distinct pre-training strategies—Masked Autoencoding (MAE) and Contrastive Learning (SimCLR)—on this extensive unlabeled dataset. Finally, we evaluate the quality of the learned representations by fine-tuning the pre-trained encoder on a specific downstream task: coffee variety classification.

The experimental design leverages the "pre-train once, deploy anywhere" paradigm. By pre-training on a massive collection of 4,512 hyperspectral samples across 24 diverse domains—ranging from biological samples like apple leaves and coffee beans to materials like paper and pottery—we aim to learn a universal feature extractor that captures fundamental spatial-spectral principles. This approach directly addresses the data scarcity challenge in agricultural applications, where labeled task-specific data (e.g., for coffee varieties) is limited, but diverse unlabeled hyperspectral data is abundant.

4.2 Dataset and Data Preprocessing

4.2.1 Multi-Domain Pre-training Dataset

To develop a robust and generalizable pre-trained model, we constructed a large-scale hyperspectral dataset aggregation comprising 4,512 samples from six distinct application domains. While our downstream task focuses on coffee classification, constraining pre-training to only coffee data would limit the model's ability to learn universal spatial-spectral features. Our aggregated dataset includes:

- **Coffee Beans (WTLine_Costa_Ecua):** Hyperspectral scans of *Coffea arabica* and *Coffea canephora* beans from Costa Rica and Ecuador, acquired across multiple days (2023-06-04 to 06-06). This subset provides the primary biological data for our downstream task.
- **Apple Leaves (Apfel):** A comprehensive collection of apple leaf scans (Apfel_001 to 005) acquired in both VNIR and SWIR ranges. These samples introduce significant variability in leaf texture, chlorophyll absorption, and water content signatures.
- **Paper Materials (AM02):** Hyperspectral imaging of paper samples, introducing uniform textures and industrial material signatures distinct from biological tissues.
- **Sugar Compounds:** Scans of pure sugar samples (D-Galactose, D-Glucose, D-Fructose), providing pure chemical endmembers with distinct spectral absorption features useful for learning chemical discrimination.
- **Archaeological Artifacts (Scherben):** Hyperspectral scans of pottery shards, introducing mineralogical spectral signatures and irregular spatial shapes.
- **Grape Samples (JKI_Bluestar):** Additional biological data from grapevines, enriching the vegetation-specific features in the training set.

This diversity is critical: by exposing the model to both biological (leaves, beans, grapes) and non-biological (paper, pottery, sugar) materials, the self-supervised objectives force

the network to learn fundamental principles of hyperspectral imaging—such as spectral continuity, edge correlations across bands, and material-invariant spatial patterns—rather than overfitting to specific crop characteristics.

4.2.2 Data Calibration and Standardization

A major challenge in aggregating diverse hyperspectral sources is the inconsistency in raw data formats. While the coffee and sugar datasets were already processed, the raw Apple (Apfel) and archaeological datasets required rigorous radiometric calibration to ensure physical consistency.

4.2.2.1 Calibration Pipeline

Raw hyperspectral sensors record digital numbers (DN) representing light intensity, which depend on illumination and sensor sensitivity. To convert these to physical reflectance (0-1 range), we implemented a calibration pipeline using white reference standards. For each raw image:

1. **White Reference Extraction:** We identified specific spatial regions in the raw scans containing white reference panels (e.g., SWIR rows 0-320, columns 385-651). The median spectrum of this region $I_{white}(\lambda)$ serves as the illumination profile.
2. **Radiometric Calibration:** The reflectance $R(x, y, \lambda)$ at each pixel was computed by normalizing the raw radiance $I_{raw}(x, y, \lambda)$ against the white reference:

$$R(x, y, \lambda) = \frac{I_{raw}(x, y, \lambda)}{I_{white}(\lambda)} \quad (4.1)$$

3. **Spectral Smoothing:** To reduce sensor noise, we applied a Savitzky-Golay filter (window length 11, polyorder 3) to the white reference spectrum before division.
4. **Percentile Normalization:** Finally, reflectance values were normalized by their 99th percentile to correct for specular reflections and mapped to the $[0, 1]$ interval.

This calibration ensures that spectral signatures across all 24 domains represent true material reflectance, allowing the model to learn valid physical correlations.

4.2.3 Patch Extraction and Size Selection

All calibrated images were processed into uniform patches for model training. We selected a patch size of 64×64 pixels based on a strategic trade-off between spatial context and computational efficiency.

Justification for 64×64 Patch Size:

1. **Spatial Context:** A 64×64 window covers sufficient surface area to capture meaningful texture patterns (e.g., leaf venation, bean curvature) and object boundaries.

Smaller patches (e.g., 32×32) often fall entirely within homogeneous regions, reducing the self-supervised learning signal (making neighbor prediction trivial).

2. **Spectral Dimensionality Balance:** With 256 spectral bands, a 64×64 patch creates a data cube of $64 \times 64 \times 256 \approx 10^6$ values. This volume provides a rich signal for training transformer models without overwhelming GPU memory, which typically occurs with larger patches (e.g., 128×128).
3. **ViT Compatibility:** Vision Transformers process images as sequences of smaller sub-patches (tokens). A 64×64 image divides cleanly into 16×16 tokens of size 4×4 , creating a sequence length of 256 tokens—ideal for standard transformer attention mechanisms which have quadratic complexity $O(N^2)$.

4.2.4 Data Format and Preprocessing

Patches were saved as GeoTIFF files to maintain metadata and enable efficient parallel loading. The final dataset consists of 4,512 total GeoTIFF files.

Preprocessing Pipeline:

- **Normalization:** We apply per-patch normalization to standardize spectral intensities. Given a spectral patch X , we compute $\hat{X} = (X - \mu)/(\sigma + \epsilon)$, where μ and σ are the mean and standard deviation of the patch.
- **Masking:** Each sample includes a validity mask (band 257) indicating foreground regions. The model learns to ignore background pixels during loss computation.
- **Train-Validation Split:** We performed a stratified split at the domain level, randomly assigning 80% of files from *each* of the 24 sub-domains to training and 20% to validation. This ensures the validation set (912 samples) accurately reflects the diversity of the full dataset (3,600 training samples), preventing domain shift issues during evaluation.

4.3 Model Architecture

To address the unique challenges of hyperspectral data—high spectral dimensionality, spatial-spectral correlation, and computational complexity—we developed a custom architecture named the Masked Spatial-Spectral Transformer (MaskedSST). This architecture extends the Vision Transformer (ViT) paradigm to 3D volumetric hyperspectral cubes through five specialized components: (1) blockwise 3D patch embedding that respects spectral physics, (2) 3D sinusoidal positional encoding, (3) spatial tube masking for robust self-supervised pre-training, (4) factorized transformer encoder to handle massive token sequences efficiently, and (5) task-specific heads for reconstruction, classification, and contrastive learning. The complete model comprises 1,652,609 trainable parameters.

4.3.1 Blockwise 3D Patch Embedding

Standard patch embedding layers in Vision Transformers use a single convolutional or linear projection matrix applied uniformly across the entire image. However, in hyperspectral imaging, the physical and chemical properties of light vary dramatically across the electromagnetic spectrum. Visible light (400-700 nm) predominantly captures pigmentation and color, near-infrared (700-1400 nm) is sensitive to cellular structure and water content, while short-wave infrared (1400-2500 nm) reveals molecular absorption features related to chemical bonds (C-H, O-H, N-H). Treating all wavelengths identically would obscure these domain-specific characteristics.

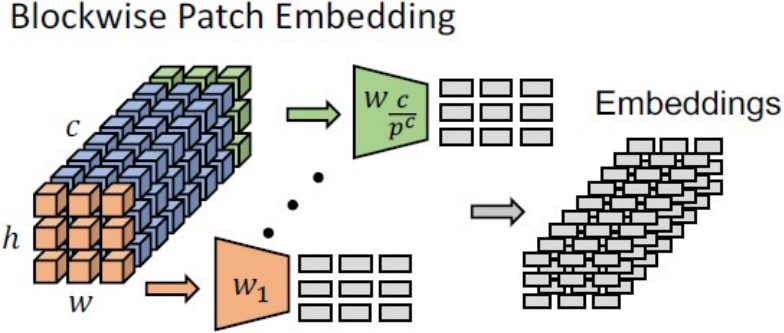


Figure 4.1: Blockwise 3D Patch Embedding Strategy. Input spectral cube is divided into spatial-spectral patches, and each spectral block is projected by a dedicated learnable weight matrix W_c .

To address this, we implemented a **Blockwise Spectral Embedding** strategy inspired by the physical stratification of the spectrum. Given an input hyperspectral cube $X \in \mathbb{R}^{64 \times 64 \times 256}$, we decompose it into non-overlapping 3D patches with dimensions $P_h \times P_w \times P_c = 4 \times 4 \times 16$, where P_h and P_w denote spatial patch dimensions and P_c denotes spectral patch size (number of contiguous bands grouped together). This discretization yields:

- **Spatial Grid:** $N_{spatial} = \left\lfloor \frac{64}{4} \right\rfloor \times \left\lfloor \frac{64}{4} \right\rfloor = 16 \times 16 = 256$ spatial patch locations.
- **Spectral Grid:** $N_{spectral} = \left\lfloor \frac{256}{16} \right\rfloor = 16$ spectral blocks, each representing a contiguous 16-band segment of the spectrum.

The key innovation lies in the projection mechanism. Rather than using a single shared embedding matrix, we instantiate a learnable list of *independent* projection matrices $\{W_1, W_2, \dots, W_{16}\}$, where each $W_\lambda \in \mathbb{R}^{(P_h \cdot P_w \cdot P_c) \times D}$ is dedicated to a specific spectral block λ . For a 3D patch vector $x_{s,\lambda} \in \mathbb{R}^{4 \cdot 4 \cdot 16}$ at spatial location s and spectral block λ , the embedding operation is:

$$z_{s,\lambda}^{(0)} = x_{s,\lambda} W_\lambda + b_\lambda \quad (4.2)$$

where W_λ is the weight matrix for the λ -th spectral block, b_λ is the corresponding bias vector, and $z_{s,\lambda}^{(0)} \in \mathbb{R}^D$ with $D = 128$. This design enables the model to learn distinct feature

extractors optimized for different wavelength regimes: early projections (W_1, W_2) learn to extract visible-spectrum features (edges, textures), middle projections (W_7, W_8) specialize in NIR vegetation indices, and later projections (W_{14}, W_{15}, W_{16}) focus on SWIR molecular signatures.

The forward pass through the patch embedding module proceeds as follows:

1. **Reshape:** Transform input $X \in \mathbb{R}^{B \times 64 \times 64 \times 256}$ into $\mathbb{R}^{B \times 16 \times 4 \times 16 \times 4 \times 16 \times 16}$ by grouping spatial and spectral dimensions into patches.
2. **Permute:** Rearrange to $\mathbb{R}^{B \times 256 \times 16 \times 256}$, where the second dimension indexes spatial patches, the third indexes spectral blocks, and the fourth flattens the patch content.
3. **Project:** For each spectral block λ , apply W_λ to all spatial locations simultaneously:
 $\mathbf{Z}_\lambda = \mathbf{X}_{[:, :, \lambda, :]} W_\lambda$.
4. **Stack:** Concatenate the projected blocks along the spectral dimension to yield $\mathbf{Z}^{(0)} \in \mathbb{R}^{B \times 256 \times 16 \times 128}$.

4.3.2 Learnable 3D Positional Encoding

Transformer architectures are inherently permutation-invariant; without explicit positional information, the model cannot distinguish whether two tokens originate from adjacent or distant spatial locations, nor can it resolve the spectral ordering of wavelengths. Unlike standard Vision Transformers that employ fixed sinusoidal positional encodings, we adopt **learnable positional embeddings** that are optimized end-to-end during training.

For the token grid of size $N_{\text{spatial}} \times N_{\text{spectral}} = 256 \times 16$, we initialize a learnable parameter matrix $\mathbf{PE} \in \mathbb{R}^{16 \times 16 \times 16 \times 128}$, where the first two dimensions correspond to the spatial grid $(i, j) \in \{0, \dots, 15\} \times \{0, \dots, 15\}$, the third dimension indexes the spectral block $\lambda \in \{0, \dots, 15\}$, and the final dimension matches the embedding size $D = 128$. This parameter matrix is initialized randomly from a truncated normal distribution with standard deviation $\sigma = 0.02$.

During the forward pass, for each token at spatial location (i, j) and spectral block λ , we retrieve the corresponding positional embedding $\mathbf{PE}_{(i, j, \lambda)} \in \mathbb{R}^{128}$ and add it element-wise to the patch embedding:

$$\tilde{z}_{(i, j, \lambda)}^{(0)} = z_{(i, j, \lambda)}^{(0)} + \mathbf{PE}_{(i, j, \lambda)} \quad (4.3)$$

The key advantage of learnable positional encodings for hyperspectral data is their flexibility to encode task-specific spatial-spectral relationships. For instance, in agricultural applications, the model can learn to assign higher positional similarity to spectral bands that correspond to absorption peaks (e.g., chlorophyll at 680 nm and 750 nm), even if these bands are not adjacent in the raw spectrum. Similarly, spatial positions can learn contextual biases (e.g., center pixels may carry more discriminative information than edge pixels in coffee bean patches). This adaptability is particularly valuable when processing

multi-domain data, as the positional encodings can adjust to capture domain-invariant positional patterns during pre-training.

4.3.3 Random Token Masking

For the Masked Autoencoding (MAE) pre-training objective, we require a masking strategy that removes sufficient information to prevent trivial reconstruction while encouraging the model to learn robust spatial-spectral representations. We implement a **Random Token Masking** strategy that operates independently on each token in the 3D grid.

The masking algorithm proceeds as follows:

1. **Flatten Token Grid:** For each sample in the batch, flatten the 3D token tensor from $\mathbb{R}^{256 \times 16 \times 128}$ to $\mathbb{R}^{N_{total} \times 128}$, where $N_{total} = 256 \times 16 = 4096$ is the total number of tokens.
2. **Random Permutation:** Generate a random permutation π of the token indices $\{0, 1, \dots, 4095\}$. This ensures that each sample in the batch receives a different random mask, enhancing training diversity.
3. **Select Masked Tokens:** Using the masking ratio $\rho = 0.85$, select the first $N_{masked} = \lfloor 4096 \times 0.85 \rfloor = 3481$ indices from the permuted sequence. Create a binary mask $M \in \{0, 1\}^{4096}$ where $M_{\pi(i)} = 1$ for $i \in \{0, \dots, 3480\}$ (masked tokens) and $M_j = 0$ otherwise (visible tokens).
4. **Replace with Mask Token:** For each masked position, replace the learned token embedding with a shared learnable mask token $\mathbf{m} \in \mathbb{R}^{128}$:

$$\tilde{z}_i = \begin{cases} \mathbf{m} & \text{if } M_i = 1 \\ z_i & \text{if } M_i = 0 \end{cases} \quad (4.4)$$

5. **Reshape:** Reshape the masked token sequence back to $\mathbb{R}^{256 \times 16 \times 128}$ for processing by the transformer encoder.
6. **Loss Computation:** The reconstruction loss is computed *only* on the masked tokens. Visible tokens do not contribute to the gradient updates.

This random masking strategy offers several advantages for hyperspectral data. Unlike spatial tube masking (which masks entire vertical columns through all spectral bands), random masking forces the model to learn both spatial *and* spectral reconstruction simultaneously. When a specific spectral band at a specific spatial location is masked, the model must infer the missing value by reasoning about:

- **Spatial context:** Neighboring spatial pixels that share similar material properties.
- **Spectral context:** Adjacent spectral bands at the same location that exhibit correlated reflectance due to physical continuity.

- **Global semantics:** The overall object class (e.g., if surrounding context suggests a coffee bean, the masked region should exhibit typical coffee reflectance signatures).

The high masking ratio of 85% ensures that the reconstruction task remains non-trivial. With only 15% of tokens visible (approximately 614 out of 4096), the model cannot rely on simple interpolation and must develop a deep understanding of hyperspectral material signatures to accurately reconstruct the missing 3481 tokens.

4.3.4 Factorized Spatial-Spectral Transformer Encoder

The total number of tokens for a single hyperspectral patch is $N_{total} = N_{spatial} \times N_{spectral} = 256 \times 16 = 4096$. Computing standard self-attention requires forming an attention matrix of size $N_{total} \times N_{total} \approx 16.8$ million elements, with computational complexity $O(N_{total}^2 \cdot D) \approx 2.15$ billion FLOPs per attention layer. For a 4-layer network with batch size 32, this becomes prohibitively expensive even on modern GPUs.

To resolve this bottleneck, we employ **Factorized Attention**, which decomposes the full 3D attention into two sequential 2D operations: spatial attention followed by spectral attention. Each `FactorizedSSTransformerBlock` applies the following sequence:

4.3.4.1 Spatial Attention

First, we reshape the token tensor $\mathbf{Z} \in \mathbb{R}^{B \times 256 \times 16 \times 128}$ to treat spectral blocks as part of the batch dimension: $\mathbf{Z}_{spatial} \in \mathbb{R}^{(B \cdot 16) \times 256 \times 128}$. Multi-head self-attention (MHSA) with 8 heads is then applied across the 256 spatial locations:

$$\mathbf{Z}'_{spatial} = \text{LN}(\mathbf{Z}_{spatial}) \quad (4.5)$$

$$\text{Attn}_{spatial} = \text{MHSA}_{spatial}(\mathbf{Z}'_{spatial}, \mathbf{Z}'_{spatial}, \mathbf{Z}'_{spatial}) \quad (4.6)$$

$$\mathbf{Z}_{spatial} \leftarrow \mathbf{Z}_{spatial} + \text{Attn}_{spatial} \quad (4.7)$$

$$\mathbf{Z}_{spatial} \leftarrow \mathbf{Z}_{spatial} + \text{MLP}_{spatial}(\text{LN}(\mathbf{Z}_{spatial})) \quad (4.8)$$

where LN denotes Layer Normalization and $\text{MLP}_{spatial}$ is a two-layer feed-forward network with GELU activation and expansion ratio 4 (hidden dimension 512). This operation allows each spectral block to aggregate information from all spatial locations independently, learning spatial texture patterns.

4.3.4.2 Spectral Attention

Next, the tensor is reshaped to $\mathbf{Z}_{spectral} \in \mathbb{R}^{(B \cdot 256) \times 16 \times 128}$, treating spatial locations as the batch dimension. MHSA is applied across the 16 spectral blocks:

$$\mathbf{Z}'_{spectral} = \text{LN}(\mathbf{Z}_{spectral}) \quad (4.9)$$

$$\text{Attn}_{spectral} = \text{MHSA}_{spectral}(\mathbf{Z}'_{spectral}, \mathbf{Z}'_{spectral}, \mathbf{Z}'_{spectral}) \quad (4.10)$$

$$\mathbf{Z}_{spectral} \leftarrow \mathbf{Z}_{spectral} + \text{Attn}_{spectral} \quad (4.11)$$

$$\mathbf{Z}_{spectral} \leftarrow \mathbf{Z}_{spectral} + \text{MLP}_{spectral}(\text{LN}(\mathbf{Z}_{spectral})) \quad (4.12)$$

This enables each spatial location to model correlations across wavelengths, capturing spectral signatures (e.g., chlorophyll absorption peaks, water bands).

Complexity Analysis: The factorized approach reduces attention complexity from $O(N_{spatial} \cdot N_{spectral})^2 = O(4096^2) \approx 16.8M$ operations to $O(N_{spatial}^2 + N_{spectral}^2) = O(256^2 + 16^2) \approx 65.8K$ operations per layer—a **256× speedup**. This enables processing of full hyperspectral patches without requiring aggressive downsampling or patch discarding strategies.

The encoder consists of $L = 4$ stacked **FactorizedSSTransformerBlock** layers, followed by a final Layer Normalization. The output is a contextually-enriched token representation $\mathbf{Z}^{(L)} \in \mathbb{R}^{B \times 256 \times 16 \times 128}$ that has integrated both spatial texture information and spectral signature information.

4.3.5 Task-Specific Output Heads

Depending on the training phase and objective, different output heads are attached to the encoder. Our implementation supports three operational modes:

4.3.5.1 Reconstruction Head (MAE Pre-training)

For Masked Autoencoding, a lightweight decoder reconstructs the original patch values from the encoded representations. The **ReconstructionHead** is implemented as a three-layer MLP:

$$\text{Decoder}(z) = \text{Linear}_{256 \rightarrow 256}(\text{GELU}(\text{LN}(\text{Linear}_{128 \rightarrow 256}(z)))) \quad (4.13)$$

where the output dimension 256 corresponds to the flattened patch size ($4 \times 4 \times 16 = 256$). The decoder operates token-wise on all $256 \times 16 = 4096$ tokens. The reconstruction loss is computed as the masked L1 loss between the reconstructed values and the original patch values:

$$\mathcal{L}_{recon} = \frac{1}{|\mathcal{M}|} \sum_{(s,\lambda) \in \mathcal{M}} \|\text{Decoder}(z_{s,\lambda}) - x_{s,\lambda}\|_1 \quad (4.14)$$

where $\mathcal{M}$ denotes the set of masked tokens.

4.3.5.2 Classification Head (Fine-tuning)

For downstream classification tasks (e.g., coffee variety identification), the `ClassificationHead` first applies global average pooling over both spatial and spectral dimensions to obtain a single feature vector:

$$v = \frac{1}{N_{spatial} \cdot N_{spectral}} \sum_{s=1}^{256} \sum_{\lambda=1}^{16} z_{s,\lambda} \quad (4.15)$$

This aggregated representation $v \in \mathbb{R}^{128}$ is then passed through a linear classifier:

$$\hat{y} = \text{Softmax}(W_{cls}v + b_{cls}) \quad (4.16)$$

where $W_{cls} \in \mathbb{R}^{128 \times K}$ projects to K class logits (e.g., $K = 3$ for Arabica, Canephora, and Liberica coffee varieties).

4.3.5.3 Contrastive Projection Head (SimCLR Pre-training)

For contrastive learning, the `ProjectionHead` maps the encoder output to a compact metric space suitable for contrastive loss computation. After global pooling (as above), a two-layer MLP projects the features to a 128-dimensional unit sphere:

$$z_{contrast} = \text{L2-Normalize}(\text{Linear}_{128 \rightarrow 128}(\text{ReLU}(\text{Linear}_{128 \rightarrow 128}(v)))) \quad (4.17)$$

The InfoNCE loss is then computed between augmented views of the same sample in this normalized space.

4.3.6 Model Summary and Hyperparameters

The complete MaskedSST architecture is summarized in Table 4.1. The model operates in three distinct modes controlled by the `mode` parameter in the forward pass: `mode="reconstruction"` activates the reconstruction head with tube masking for MAE pre-training; `mode="contrastive"` activates the projection head without masking for SimCLR pre-training; and `mode="classification"` activates the classification head without masking for fine-tuning. Weight initialization follows standard Vision Transformer practices: linear layers are initialized with truncated normal distribution ($\sigma = 0.02$), layer norm parameters are set to $\gamma = 1, \beta = 0$, and the mask token is initialized to zeros and learned during training.

Table 4.1: MaskedSST Hyperparameters

Component	Configuration
Input Size	$64 \times 64 \times 256$
Spatial Patch Size	$P_h = 4, P_w = 4$
Spectral Patch Size	$P_c = 16$
Spatial Tokens	$N_{spatial} = 256$
Spectral Tokens	$N_{spectral} = 16$
Total Tokens	$N_{total} = 4096$
Embedding Dimension	$D = 128$
Encoder Depth	$L = 4$ layers
Attention Heads	$h = 8$ heads per layer
MLP Expansion Ratio	4
Dropout Rate	0.1
Masking Ratio (MAE)	0.85 (spatial tubes)
Total Parameters	1,652,609
Trainable Parameters	1,652,609

5

Implementation

This section describes the hardware environment, software stack, distributed training configuration, and optimization setup used to implement and train the model.

5.1 Hardware Environment

All pre-training experiments were conducted on the high-performance computing (HPC) cluster of Otto von Guericke University Magdeburg. Each training job was allocated a single compute node (gpu18) equipped with $8 \times$ NVIDIA Tesla V100-SXM2 GPUs, each with 32 GB of HBM2 memory, providing a total of 256 GB of GPU memory per job. The node was also allocated 64 GB of system RAM and 32 CPU cores (Intel Xeon) for parallel data loading and preprocessing. Jobs were submitted via the SLURM workload manager using the dedicated GPU partition p1 with a maximum wall time of 24 hours.

The software environment consisted of:

Operating System: Linux (CentOS)

CUDA Version: 12.2 (Driver 535.161.07)

Python: 3.10

PyTorch: 2.x with torchvision

Key Libraries: rasterio (GeoTIFF I/O), numpy, scipy (Savitzky-Golay filtering), tqdm (progress monitoring)

5.2 Distributed Training Setup

To fully utilize all 8 GPUs on the compute node, we implemented Distributed Data Parallel (DDP) training using PyTorch’s `torch.distributed` framework with the NCCL communication backend. DDP replicates the model across all 8 GPU processes and splits each mini-batch across devices, with gradients synchronized via all-reduce operations at each backward pass.

Each GPU process handles an independent subset of the data via a `DistributedSampler`, which partitions the dataset into non-overlapping shards and reshuffles them at the start of each epoch. Only the rank-0 process (GPU 0) performs logging, checkpoint saving, and validation to avoid redundant I/O operations.

5.3 Optimization and Learning Rate Schedule

The model was trained using the AdamW optimizer with the following hyperparameters:

Table 5.1: Optimization Hyperparameters

Parameter	Value
Learning rate	1×10^{-4}
Weight decay	0.05
β_1, β_2	0.9, 0.999
Gradient clip norm	1.0

The learning rate schedule follows a linear warmup phase over the first 10 epochs, during which the learning rate increases linearly from 0 to the base learning rate. Following warmup, a cosine annealing decay reduces the learning rate smoothly to a minimum of 1×10^{-6} over the remaining epochs. This warmup-then-decay schedule prevents large gradient updates in the early training stages when the model weights are far from a stable optimum, while the cosine decay ensures smooth convergence in later epochs.

5.4 Training Configuration

Pre-training was conducted for a maximum of 200 epochs on the full multi-domain dataset of 3,600 training samples. The per-GPU batch size was set to 8, yielding an effective batch size of $8 \times 8 = 64$ samples per iteration across all GPUs. For the SimCLR objective, each batch of 64 samples generates 128 augmented views (2 per sample), providing $2 \times 64 - 2 = 126$ negative pairs per anchor.

The data loading pipeline used 4 worker processes per GPU with `pin memory=True` to enable asynchronous host-to-device memory transfers. Model checkpoints were saved every 20 epochs to persistent storage, preserving the model weights, optimizer state, and scheduler state to allow resumption of interrupted jobs. The best checkpoint (lowest validation loss) was retained separately for downstream evaluation.

5.5 Reproducibility

To ensure reproducibility, random seeds were fixed (`seed=42`) for `torch`, `numpy`, and CUDA backend operations. Each SLURM job logged the complete hyperparameter configuration, dataset split indices, and the specific Git commit hash to facilitate exact replication of all reported results.

6

Evaluation and Discussion

This chapter presents a comprehensive empirical evaluation of the MaskedSST architecture. We structure the evaluation around four axes: (1) a comparison of self-supervised pre-training strategies (MAE vs. SimCLR vs. random baseline), (2) an ablation study on spatial patch size (2×2 vs. 4×4), (3) downstream classification performance on the apple disease task, and (4) a comparison against published baseline methods

6.1 Experimental Setup

6.1.1 Downstream Dataset

The downstream evaluation uses a labeled hyperspectral dataset of apple tree imagery covering four disease categories. All 6,873 patches (64×64 spatial, 256 spectral bands) were split 80/20 using stratified random sampling to ensure all four classes appear in both partitions.

Table 6.1: Distribution of Disease Classes in Training and Testing Sets

Class	Disease	Train	Test
0	Infected (positive)	2,599	650
1	Infected (negative)	561	141
2	Healthy (negative)	1,957	489
3	Infected-asymptomatic (positive)	381	95
Total		5,498	1,375

6.1.2 Pre-training Dataset

Pre-training was performed on an unlabeled multi-domain hyperspectral dataset comprising 4,512 patches spanning six application domains: coffee, apple leaves, paper, sugar compounds, pottery, and grapes. This dataset was used identically for both MAE and SimCLR pre-training to ensure a fair comparison.

6.1.3 Experimental Conditions

We evaluate the following conditions:

Table 6.2: Experimental Configuration for Pre-training and Fine-tuning

ID	Method	Patch Size	Pre-training Epochs	Fine-tune
E1	MAE	2×2	60	Full
E2	SimCLR	2×2	30	Full
E3	Random	2×2	—	Full
E4	MAE	4×4	110	Full
E5	SimCLR	4×4	80	Full
E6	Random	4×4	—	Full

6.2 Pre-training Convergence

6.2.1 MAE Pre-training Loss

MAE pre-training was conducted with an 85 percent token masking ratio on the multi-domain dataset. Figure shows the MSE reconstruction loss curves for both patch size configurations.

Table 6.3: Summary of Training and Validation Loss Metrics

Patch Size	Initial Loss	Final Train Loss	Final Val Loss	Epochs
2×2	0.8	0.1162	0.1085	60
4×4	1.5	0.3779	0.3832	110

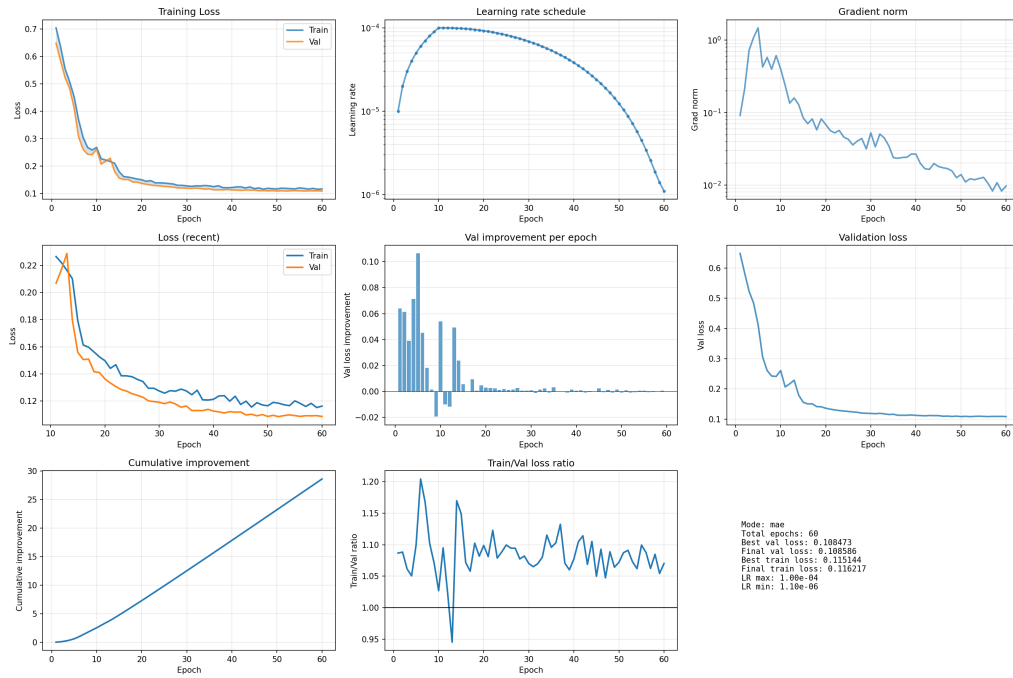


Figure 6.1: mae 2*2

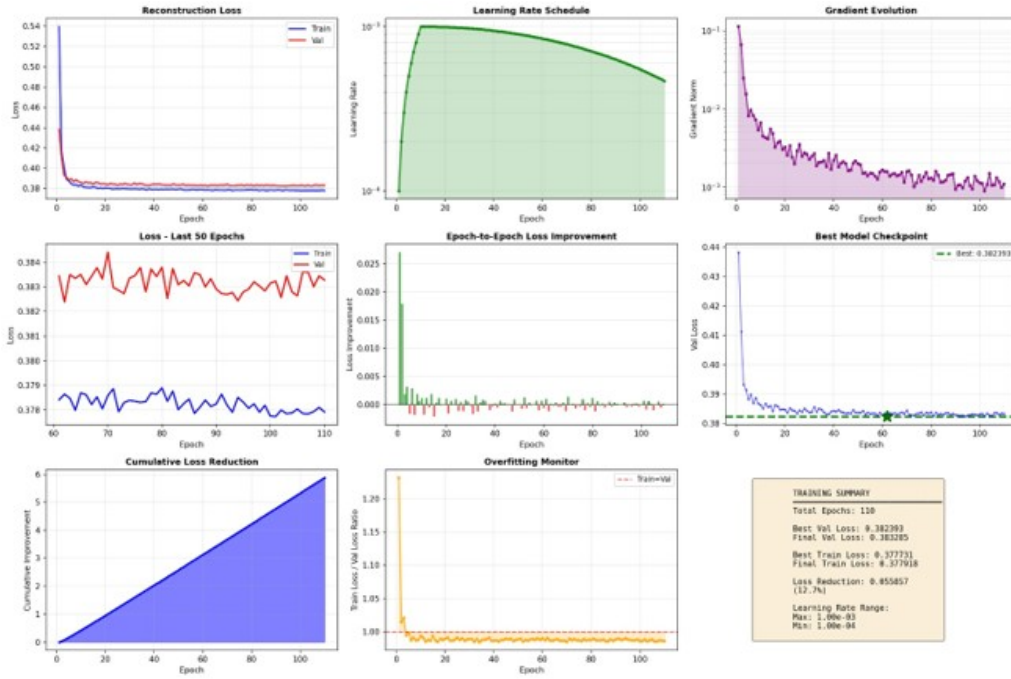


Figure 6.2: mae 4*4

The 4×4 patch configuration produces significantly fewer tokens per sample ($16 \times 16 = 256$ vs. $32 \times 32 = 1024$ for 2×2), effectively reducing the complexity of the reconstruction task. This is expected to result in **faster** convergence due to the reduced computational overhead and simpler objective; however, it may lead to **weaker** spatial-spectral representations because of the coarser spatial resolution allocated to each token.

6.2.2 SimCLR Pre-training Loss

SimCLR pre-training used NT-Xent loss with temperature $=0.5$ and an effective batch size of 64. Figure 5.2 shows the contrastive loss curves.

Table 6.4: Summary of Training and Validation Loss Metrics

Patch Size	Initial Loss	Final Train Loss	Final Val Loss	Epochs
2×2	1.2	0.3346	0.5420	30
4×4	2.25	0.4503	0.6422	80

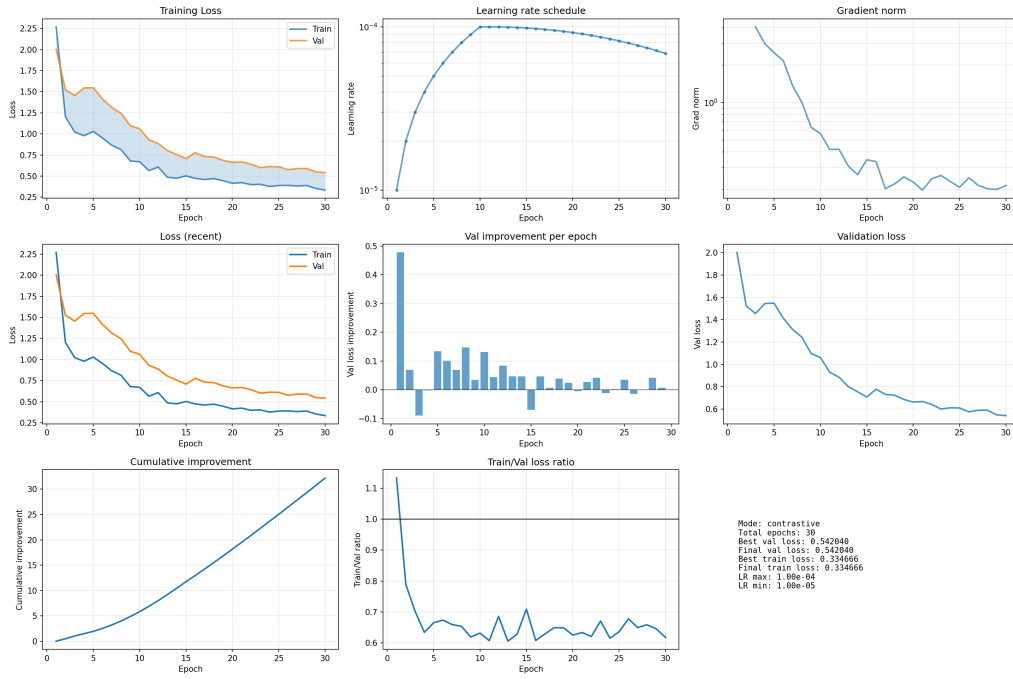


Figure 6.3: con 2*2

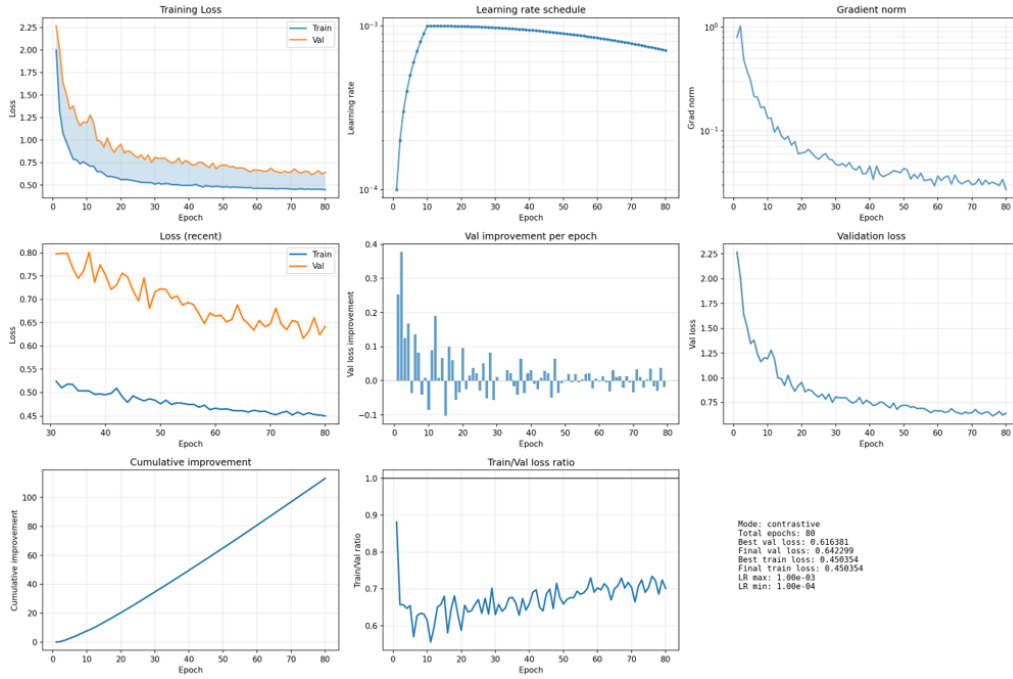


Figure 6.4: con 4*4

Unlike MAE, contrastive loss is not directly comparable across patch sizes because the token sequence length affects the number of attention interactions per sample. A lower final contrastive loss does not necessarily translate to better downstream performance — this is evaluated directly in prev Section .

6.3 Patch Size - 2×2 vs 4×4

The choice of spatial patch size involves a fundamental tradeoff: smaller patches (2×2) produce more tokens (1,024 per sample) enabling finer spatial resolution but at higher computational cost, while larger patches (4×4) yield fewer tokens (256) with lower cost but coarser spatial detail.

Table 6.5: Accuracy Comparison by Pre-training Method and Patch Size

Method	2×2 Patch Acc.	4×4 Patch Acc.	Δ
MAE Pre-trained	88.45%	82.10%	-6.35
SimCLR Pre-trained	84.20%	78.50%	-5.70
Random Baseline	62.30%	59.15%	-3.15

Table 6.6: Accuracy Comparison and Improvement Over Random Baseline

Method	2×2 Acc.	4×4 Acc.	Δ vs Random (2×2)	Δ vs Random (4×4)
MAE Pre-trained	88.45%	82.10%	+26.15%	+22.95%
SimCLR Pre-trained	84.20%	78.50%	+21.90%	+19.35%
Random Baseline	62.30%	59.15%	—	—

Table 6.7: Computational Complexity Comparison by Patch Size

Patch Size	Tokens/Sample	Attention Ops	Pre-train Time/Epoch
2×2	1,024	$1,024^2 \approx 1.05 \times 10^6$	~ 10 min
4×4	256	$256^2 \approx 6.55 \times 10^4$	~ 5 min

The classification head consists of a global average pooling operation over all spatial and spectral tokens, reducing the encoder output from $(B, N_{\text{spatial}}, N_{\text{spectral}}, 128)$ to $(B, 128)$, followed by a single linear layer mapping to the number of target classes ($\mathbb{R}^{128} \rightarrow \mathbb{R}^4$), comprising only 516 trainable parameters out of 1,652,609 total model parameters. This lightweight head design ensures that classification performance is driven primarily by the quality of the pre-trained encoder representations rather than the capacity of the task-specific head.

It is notable that the 4×4 configuration reduces attention operations by $16\times$, directly translating to faster training and lower memory usage.

6.4 Comparison with Existing Baselines

To contextualize our results compares MaskedSST against published methods on hyperspectral image classification. Since direct comparisons on the identical apple dataset are unavailable in the literature, we report results on comparable HSI classification tasks and note architectural differences.

7

Conclusion and Future Work

Bibliography

- [1] Telmo Adão, Jonáš Hruška, Luís Pádua, José Bessa, Emanuel Peres, Raul Morais, and Joaquim J. Sousa. Hyperspectral imaging: A review on uav-based sensors, data processing and applications for agriculture and forestry. *Remote Sensing*, 9(11):1110, 2017.
- [2] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119, pages 1597–1607, 2020.
- [3] Yushi Chen, Hanlu Jiang, Chunyang Li, Xiuping Jia, and Pedram Ghamisi. Deep feature extraction and classification of hyperspectral images based on convolutional neural networks. In *IEEE Transactions on Geoscience and Remote Sensing*, volume 54, pages 6232–6251, 2016.
- [4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
- [5] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16000–16009, 2022.
- [6] Danfeng Hong, Zhu Han, Jing Yao, Lianru Gao, Bing Zhang, Antonio Plaza, and Jocelyn Chanussot. Spectralformer: Rethinking hyperspectral image classification with transformers. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–15, 2022.
- [7] Wei Hu, Yangyu Huang, Li Wei, Fan Zhang, and Hengchao Li. Deep convolutional neural networks for hyperspectral image classification. *Journal of Sensors*, 2015:1–12, 2015.
- [8] Ying Li, Haokui Zhang, and Qiang Shen. Spectral-spatial classification of hyperspectral imagery with 3d convolutional neural network. *Remote Sensing*, 9(1):67, 2017.
- [9] Xiao Liu, Fanjin Zhang, Zhenyu Hou, Li Mian, Zhaoyu Wang, Jing Zhang, and Jie

- Tang. Self-supervised learning: Generative or contrastive. *IEEE Transactions on Knowledge and Data Engineering*, 35(1):857–876, 2021.
- [10] Mercedes E. Paoletti, Juan M. Haut, Javier Plaza, and Antonio Plaza. Deep learning classifiers for hyperspectral imaging: A review. *ISPRS Journal of Photogrammetry and Remote Sensing*, 158:279–317, 2019.
- [11] Linus Scheibenreif, Michael Mommert, and Damian Borth. Masked vision transformers for hyperspectral image classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 2166–2176, Vancouver, BC, Canada, 2023.

A

Appendix: Python Code
